

Ingineria programării

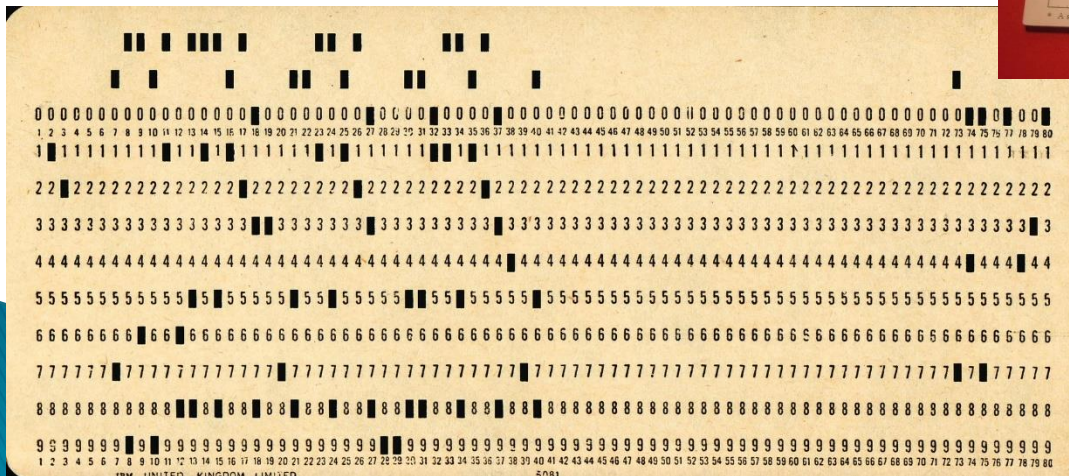
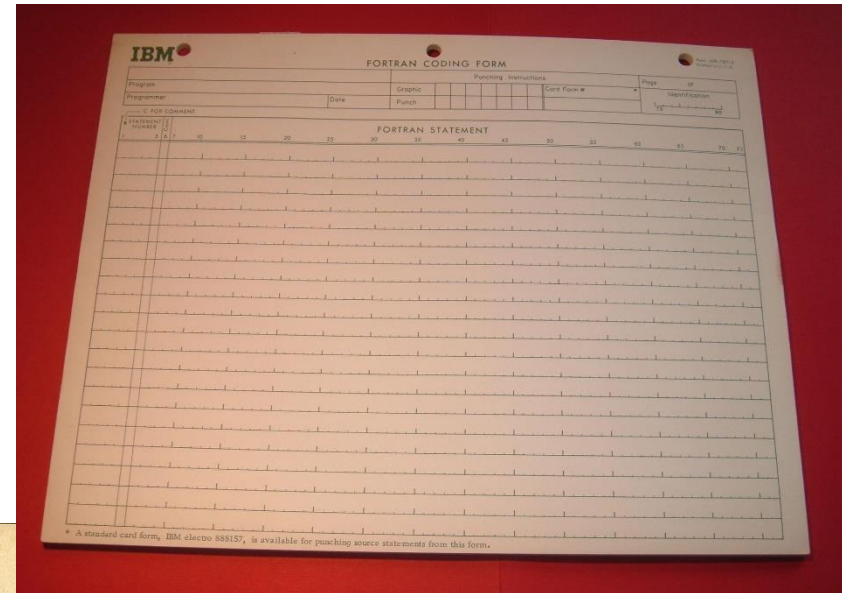
Curs 12 – 3 Iunie

Outline

- ▶ Recap
 - Software testing
 - Non-functional software testing
 - Security
 - Performance
 - Usability
 - Internationalization
 - SOLID Refactoring
 - Degradation of code
- ▶ AI Code Generation

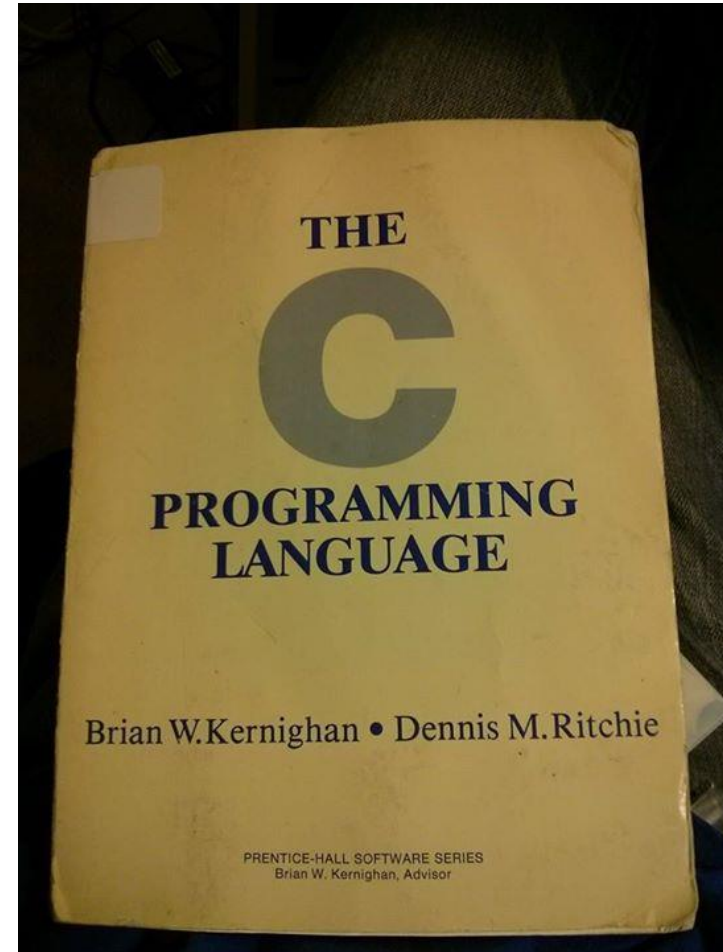
Writing Code

- ▶ Initially, code was developed on paper
 - Few computers
 - Limited memory
 - Different input devices



Writing Code

- ▶ Text editors
 - Programmers needed to know all the reserved words
 - Good programmers knew many methods in various libraries
 - You kept your programming manual close
 - You had to compile the code to see if there were any errors



Writing Code Faster

- ▶ IDE – Integrated Development Environment
 - Syntax highlighting
 - Debugging
 - **Code completion**
 - **Refactoring**
 - etc.

Writing Code Even Faster

- ▶ Expanded IDEs
 - Version control
 - Unit testing
 - Visual programming
 - **Intelligent Code Completion**
 - Uses generative AI to provide more than just simple method or name suggestions

Generative AI and Code Generation

- ▶ Generative AI is increasingly popular
 - Focused on productivity tools
- ▶ LLMs – Large Language Models
 - Algorithms using deep learning and neural networks on large sets of data (usually text)
 - Generate content similar to human content
 - Based on examples
- ▶ ChatGPT, Bard, Microsoft Copilot, Github Copilot, DALL-E, etc.

Generative Ai and Code Generation

- ▶ Improves developer productivity
 - Quickly generates boilerplate code
 - Allows developers to focus on the more demanding aspects
 - Improves job satisfaction
- ▶ Improves code quality
 - With proper code review
- ▶ Makes adoption of new tools and languages easier

Generative Ai and Code Generation

► ChatGPT example

```
public class MatrixMultiplication {  
  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        // Read dimensions of the first matrix  
        System.out.print("Enter number of rows for the first matrix: ");  
        int rows1 = scanner.nextInt();  
        System.out.print("Enter number of columns for the first matrix: ");  
        int cols1 = scanner.nextInt();  
  
        // Read dimensions of the second matrix  
        System.out.print("Enter number of rows for the second matrix: ");  
        int rows2 = scanner.nextInt();  
        System.out.print("Enter number of columns for the second matrix: ");  
        int cols2 = scanner.nextInt();  
  
        // Check if multiplication is possible  
        if (cols1 != rows2) {  
            System.out.println("Matrix multiplication is not possible with the given dimensions.");  
            return;  
        }  
  
        // Read the first matrix  
        int[][] matrix1 = new int[rows1][cols1];  
        System.out.println("Enter elements of the first matrix:");  
        for (int i = 0; i < rows1; i++) {  
            for (int j = 0; j < cols1; j++) {  
                matrix1[i][j] = scanner.nextInt();  
            }  
        }  
    }  
}
```

Generative AI and Code Generation

- ▶ Case study: Github Copilot
 - Free to use for students
 - Integrated with most major IDEs
 - Trained on public repositories in Github

Generative AI and Code Generation

- ▶ Type code and get autocomplete suggestions from AI
 - Autocompletion was among the first ways to increase productivity
 - Instead of suggesting known methods or variables, the AI agent suggests entire code snippets
 - The benefit is magnified for experienced developers, as it reduces time needed to type the code

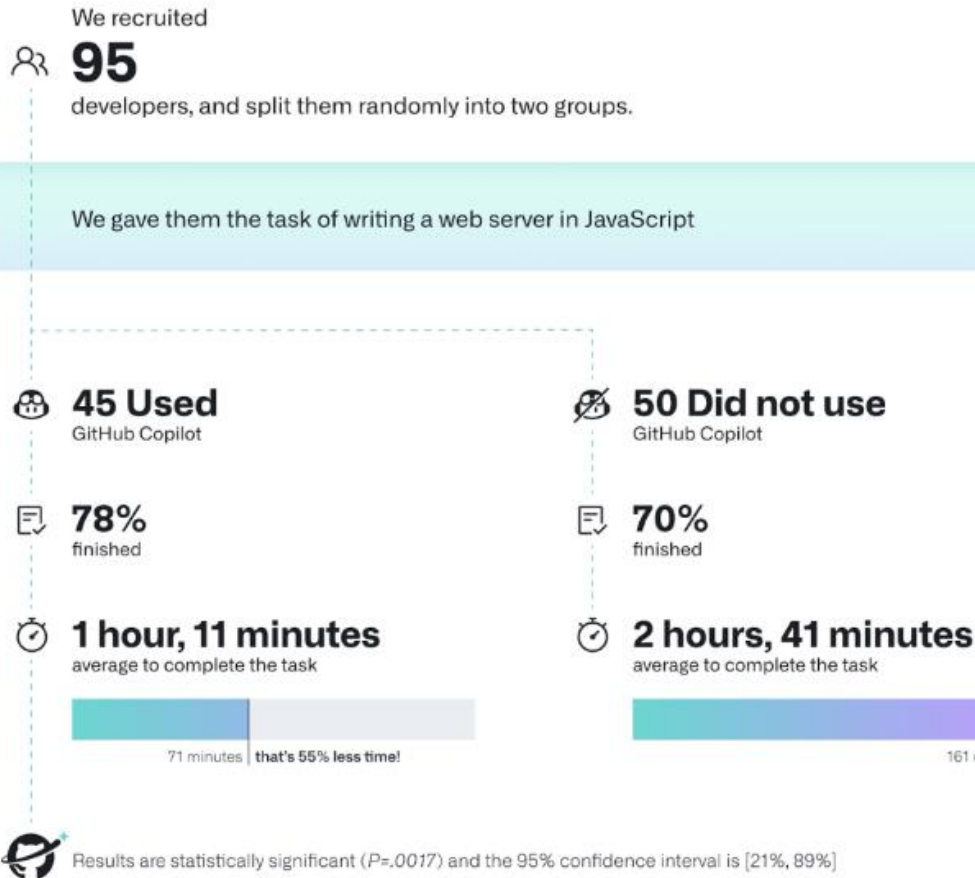
Generative AI and Code Generation

- ▶ Type code and get autocomplete suggestions from AI
 - Autocompletion was among the first ways to increase productivity
 - Instead of suggesting known methods or variables, the AI agent suggests entire code snippets
 - The benefit is magnified for experienced developers, as it reduces time needed to type the code
 - Useful for experienced developers using new programming languages

Generative AI and Code Generation

- ▶ Generate code from comments or specifications
 - Even experienced developers might have a hard time staring to write code
 - Useful in unfamiliar programming languages
 - If names of classes, methods and attributes are suggestive, the quality of the proposal is greatly increased

Github Copilot and Productivity



Code Review Is Vital

- ▶ Generated code is, usually, close to the solution
 - But code reviews are still important
 - AI generated code still has bugs
 - Discussions between developers are still irreplaceable

Advantages of Using AI Generated Code

- ▶ Increased productivity
 - Faster writing of code
 - Most visible for experienced developers
- ▶ Reduced stress and mental load
 - Unit testing is much faster
 - Code coverage made easy
- ▶ Less switching of focus
 - Reduced need to search on the web or in documentation
 - Less need to focus on details

Disadvantages of Using AI Generated Code

- ▶ Limited creativity
 - Generative AI is good at imitating but cannot innovate
- ▶ Difficulties in understanding large or complex contexts
 - Generating solutions
 - Generating tests

Example

- ▶ <https://www.pbinfo.ro/probleme/4563/suma-prefixe-pare>
 - Scrieți un program care citește un număr natural nenul n și care calculează suma S tuturor prefixelor pare ale lui n .

Example

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char n[100];
6      //printf("Introduceți un număr natural nenul: ");
7      scanf("%s", n);
8
9      long long length = strlen(n);
10     long long sum = 0;
11
12     for (int i = 0; i < length; i++) {
13         long long prefix = 0;
14         for (int j = 0; j <= i; j++) {
15             prefix = prefix * 10 + (n[j] - '0');
16         }
17         if (prefix % 2 == 0) {
18             sum += prefix;
19         }
20     }
21
22     printf("%lld\n", sum);
23     return 0;
24 }
```

Example

```
1  #include <stdio.h>
2
3  int main() {
4      unsigned long long n;
5      //printf("Introduceți un număr natural nenul: ");
6      scanf("%llu", &n);
7
8      unsigned long long sum = 0;
9      unsigned long long prefix = 0;
10     unsigned long long factor = 1;
11     unsigned long long temp = n;
12
13     // Calculăm factorul corespunzător celei mai semnificative cifre
14     while (temp > 0) {
15         factor *= 10;
16         temp /= 10;
17     }
18     factor /= 10;
19
20     // Construim și verificăm prefixele
21     while (factor > 0) {
22         prefix = (n / factor) * factor; // in varianta generata
23         if (prefix % 2 == 0) {
24             sum += prefix;
25         }
26         factor /= 10;
27     }
28
29     printf("%llu\n", sum);
30     return 0;
31 }
```

Example

```
1  #include <stdio.h>
2
3  int main() {
4      long long unsigned n;
5      //printf("Introduceti un numar natural nenul: ");
6      scanf("%llu", &n);
7
8      long long unsigned S = 0;
9      long long unsigned prefix = 0;
10     long long unsigned temp = n;
11
12     // Calcularea sumei prefixelor pare
13     while (temp > 0) {
14         prefix = prefix * 10 + temp % 10;
15         temp /= 10;
16     }
17
18     long long unsigned reversed_prefix = 0;
19     while (prefix > 0) {
20         reversed_prefix = reversed_prefix * 10 + prefix % 10;
21         prefix /= 10;
22         if (reversed_prefix % 2 == 0) {
23             S += reversed_prefix;
24         }
25     }
26
27     printf("%llu\n", S);
28     return 0;
29 }
```

Example

```
1  #include <stdio.h>
2
3  int main() {
4      long long unsigned n;
5      //printf("Introduceti un numar natural nenul: ");
6      scanf("%llu", &n);
7
8      long long unsigned S = 0;
9      long long unsigned prefix = 0;
10     long long unsigned temp = n;
11
12     // Calcularea sumei prefixelor pare
13     while (temp > 0) {
14         prefix = prefix * 10 + temp % 10; //loses trailing zeros
15         temp /= 10;
16     }
17
18     long long unsigned reversed_prefix = 0;
19     while (prefix > 0) {
20         reversed_prefix = reversed_prefix * 10 + prefix % 10;
21         prefix /= 10;
22         if (reversed_prefix % 2 == 0) {
23             S += reversed_prefix;
24         }
25     }
26
27     printf("%llu\n", S);
28     return 0;
29 }
```

Using Generative AI in Software Engineering

- ▶ genAI is a new tool in our toolbox
- ▶ Relieves developers from the repetitive part of coding
- ▶ Do NOT use generative AI based code without proper consideration and care
 - Security: encrypted transmission, access controls, periodic security checks and improvements
 - Privacy: user privacy and data handling
 - Compliance to industry requirements

Using Generative AI in Software Engineering

- ▶ All code, both written by humans or generates automatically, is prone to bugs and faults
- ▶ Use it to increase productivity, not replace creativity

Bibliography

- ▶ <https://github.blog/2022-09-07-research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/>
- ▶ <https://github.blog/2024-02-22-how-ai-code-generation-works/>
- ▶ Special thanks to Florin Olariu